

PAWLENX · 2026-05-17

Engineering Flow Diagrams

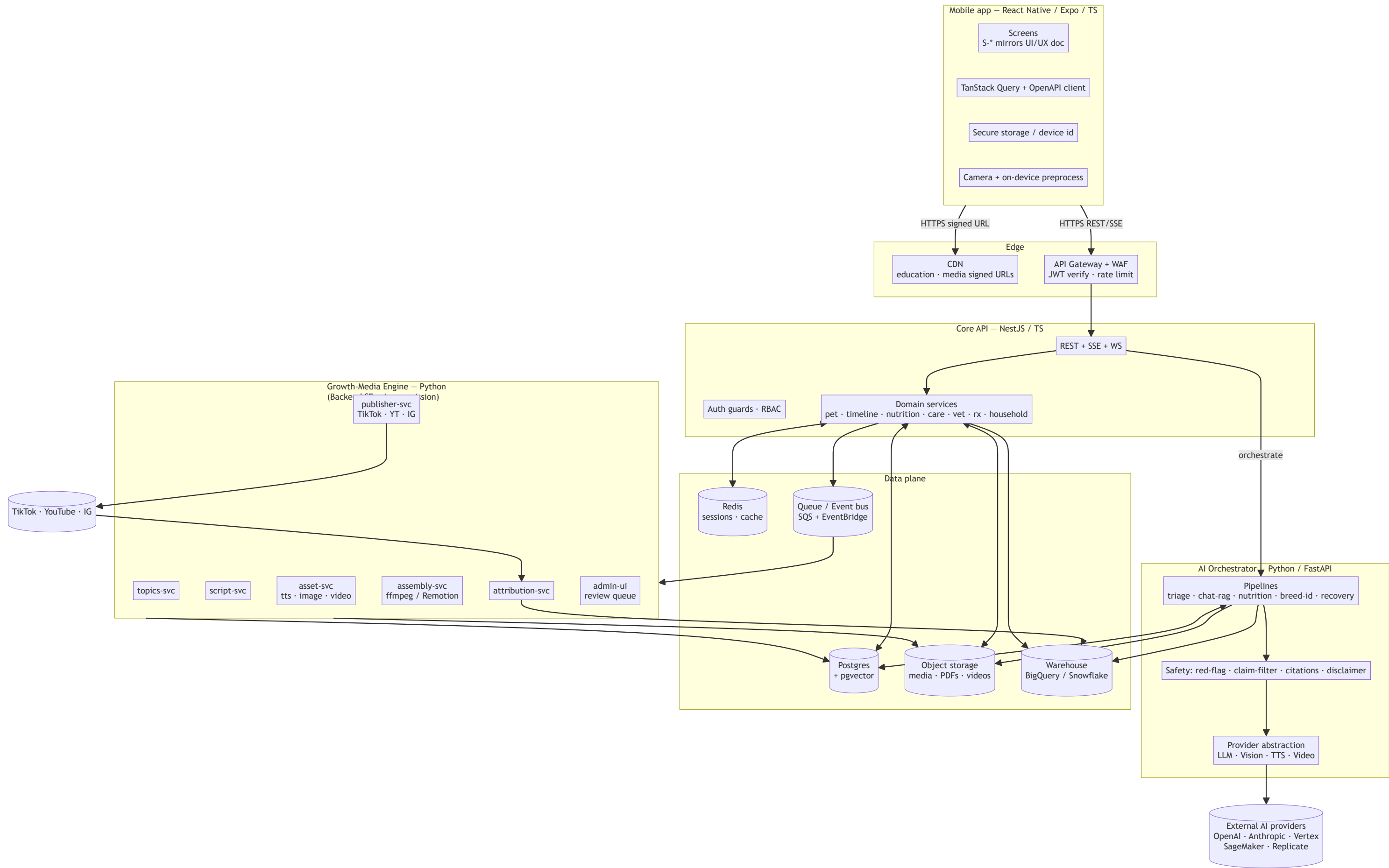
System architecture · APIs · data · AI safety · delivery

13 diagrams · each followed by a description page that names every node.

Contents

Each diagram is on its own large page; a description page follows it where node labels need explanation.

1.	High-level system architecture	8.	Authentication & authorisation flow
2.	Mobile app internal structure (mirrors Screen IDs)	9.	Growth-media content engine (Backend SE primary mission)
3.	Diagnosis request flow (end-to-end, server-side)	10.	Environments, CI/CD & release flow
4.	Chat (RAG) flow	11.	Phased delivery roadmap (MVP → V1 → V2)
5.	AI safety pipeline (mandatory on every AI output)	12.	Telemetry pipeline
6.	Async job & event topology	13.	Bidirectional handoff contract (design ↔ engineering)
7.	Data model entity relationships (simplified ERD)		



High-level system architecture

Five logical zones: mobile client, edge, core API, AI orchestrator, data plane, plus a dedicated growth-media engine (per the Backend SE JD).

CLIENT

Screens (S-* mirrors UI/UX doc)

React Native / Expo / TypeScript. Routes are 1:1 with the screen IDs in the design doc.

TanStack Query + OpenAPI client

Generated client from the same OpenAPI spec the backend serves; cache + retry.

Secure storage / device id

Keychain / Keystore for tokens; persistent device id for telemetry.

Camera + on-device preprocess

Quality check, crop, EXIF strip — before upload.

EDGE

CDN

Serves education media and short-form videos via signed URLs.

API Gateway + WAF

TLS termination, JWT verification, rate limiting, OWASP rules.

CORE API (NESTJS / TS)

REST + SSE + WS

REST for CRUD, SSE for streaming AI tokens, WS reserved for live presence.

Auth guards · RBAC

Pet → household → role resolution on every authed request.

Domain services

pet, timeline, nutrition, care, vet, rx, household — one module each.

AI ORCHESTRATOR (PYTHON / FASTAPI)

Pipelines

triage, chat-rag, nutrition, breed-id, recovery — each is a versioned pipeline.

Safety module

Red-flag rules, claim filter, citation injection, disclaimer footer.

Provider abstraction

Swap LLM/vision/TTS/video vendors without touching call sites.

DATA PLANE

Postgres + pgvector

OLTP + embeddings for RAG and pet memory.

Redis

Sessions, hot cache, idempotency keys.

Object storage (S3)

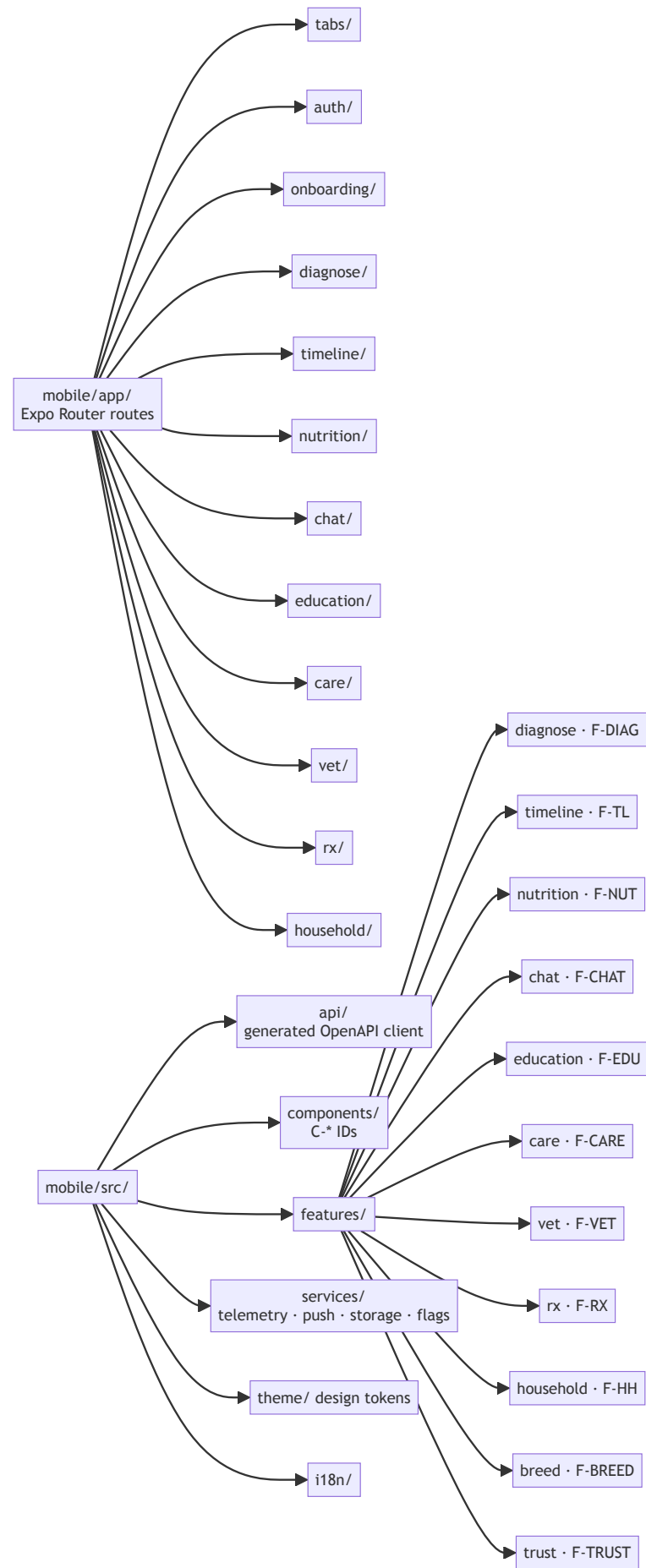
Media uploads, PDFs, rendered videos.

Queue / Event bus (SQS + EventBridge)

Decouples scan execution, scheduling, content engine, billing.

Warehouse (BigQuery / Snowflake)

Analytics and attribution.



Mobile app internal structure (mirrors Screen IDs)

Folder layout. Every route in app/ corresponds to one or more screen IDs in the design doc; every feature folder owns one feature ID (F-*).

TOP-LEVEL FOLDERS

mobile/app/

Expo Router file-system routes — one folder per major surface (tabs, auth, onboarding, etc.).

mobile/src/api/

Auto-generated OpenAPI client. Never edited by hand.

mobile/src/components/

Re-usable UI components, each with a C-* ID (e.g. C-AI-RESULT).

mobile/src/features/

One folder per feature ID — owns state, hooks, screens for that feature.

mobile/src/services/

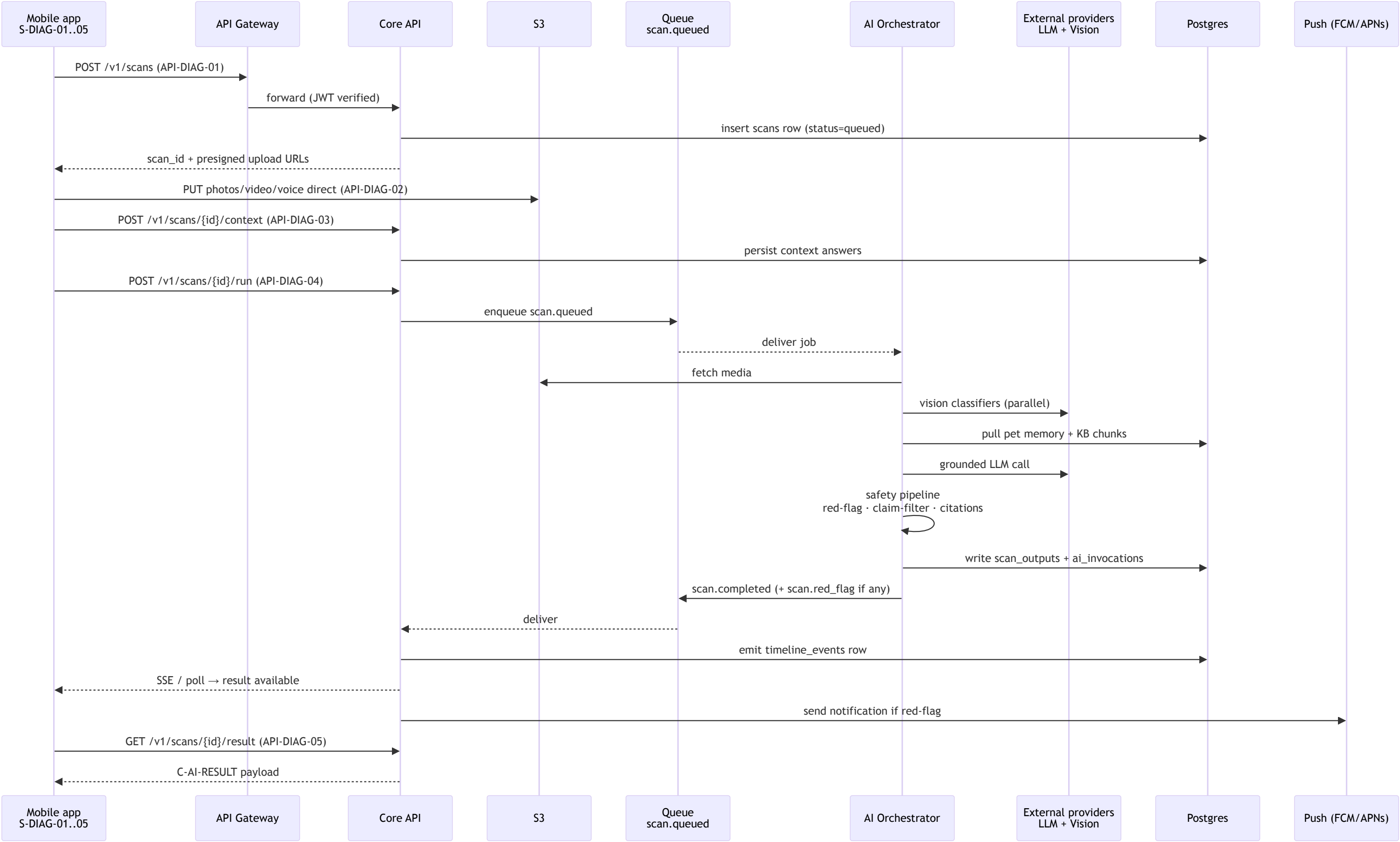
Telemetry, push, secure storage, feature flags — non-UI side effects.

mobile/src/theme/

Design tokens (colours, spacing, type scale).

mobile/src/i18n/

Translation catalogs.



Diagnosis request flow (end-to-end, server-side)

Sequence diagram. Direct-to-S3 uploads, queued AI job, SSE notification when done, separate push for red-flags. All AI work is logged in ai_invocations.

PARTICIPANTS

Mobile app

Screens S-DIAG-01..05 drive this sequence.

API Gateway

JWT verification + rate limit on every call.

Core API

Owns the scans table, emits events, returns presigned URLs.

S3

Receives the actual media — never proxied through the API.

Queue (scan.queued / scan.completed / scan.red_flag)

Decouples the user request from the long-running AI work.

AI Orchestrator

Pulls the job, runs the pipeline, writes results, emits completion event.

External providers

Vision classifiers and grounded LLM call, both in parallel where possible.

Push (FCM/APNs)

Only fired when scan.red_flag is emitted — bypasses quiet hours.

ENDPOINTS

POST /v1/scans (API-DIAG-01)

Creates the scan row, returns scan_id and presigned upload URLs.

PUT to S3 (API-DIAG-02)

Direct upload of each media artefact.

POST /v1/scans/{id}/context (API-DIAG-03)

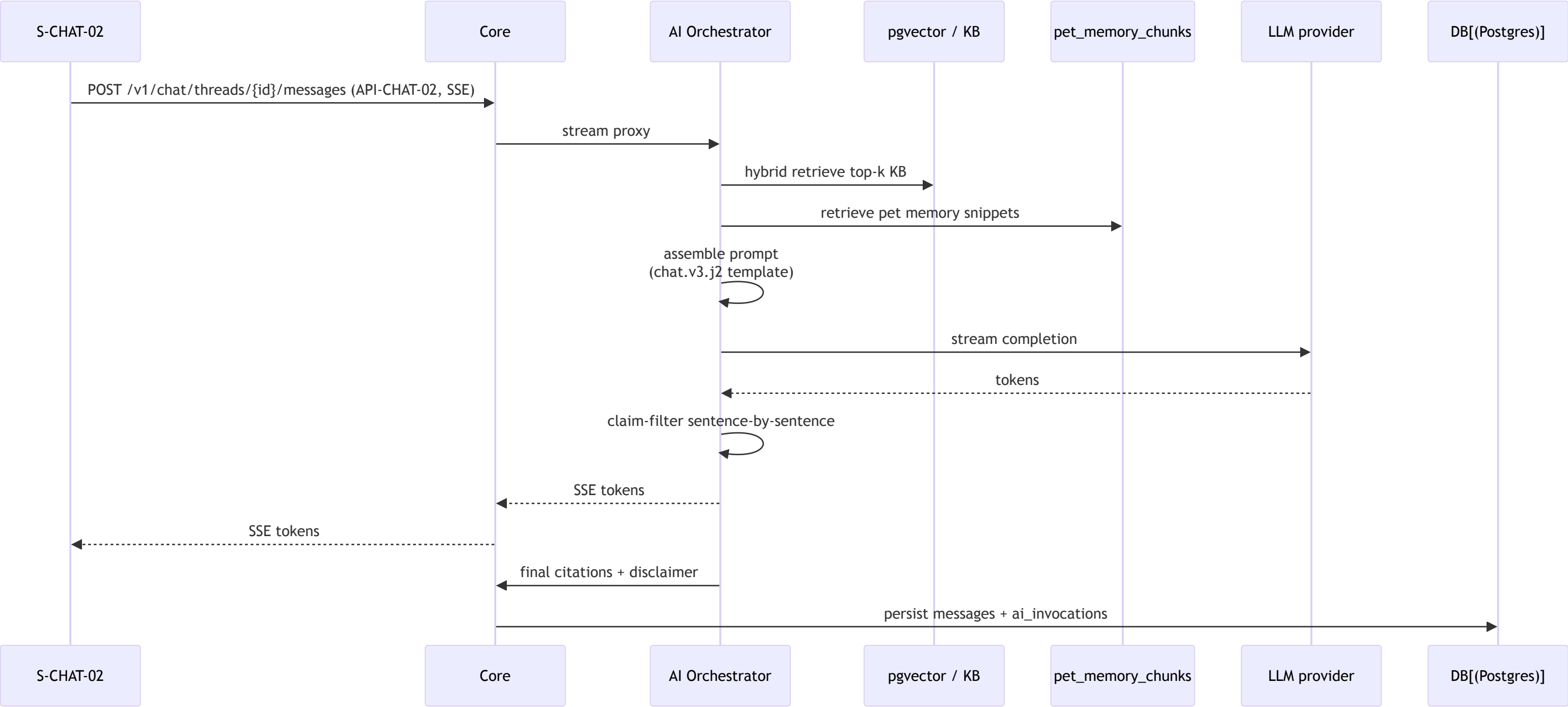
Stores adaptive context answers.

POST /v1/scans/{id}/run (API-DIAG-04)

Queues the AI job; returns immediately.

GET /v1/scans/{id}/result (API-DIAG-05)

Returns the C-AI-RESULT payload once status=completed.



Chat (RAG) flow

Sequence diagram for a chat turn. Hybrid retrieval against pet memory and the curated KB; tokens streamed over SSE; sentence-level claim filtering runs concurrently.

PARTICIPANTS

Mobile app (S-CHAT-02)

Sends the user message, renders streamed tokens.

Core API

Proxies the SSE stream; persists messages and AI invocations.

AI Orchestrator

Owns the prompt template (chat.v3.j2) and the streaming response.

pgvector / KB

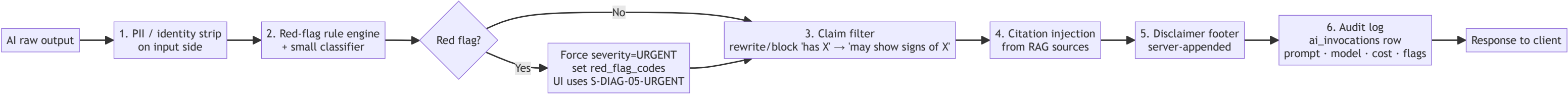
Hybrid keyword + vector retrieval over the curated knowledge base.

pet_memory_chunks

Per-pet summarised history used to ground answers.

LLM provider

Streams tokens; AI orchestrator runs the safety pipeline concurrently.



AI safety pipeline (mandatory on every AI output)

Six stages. Cannot be skipped or bypassed by any pipeline. Output is only returned to the client after stage 5; stage 6 writes the audit row.

STAGES

1 — PII / identity strip (input side)

Names, addresses and contact info removed from the prompt before it leaves our infra.

2 — Red-flag rule engine + small classifier

Term list + lightweight classifier flag conditions that need urgent attention.

3 — Claim filter

Rewrites or blocks definitive medical claims ("has X" → "may show signs of X").

4 — Citation injection

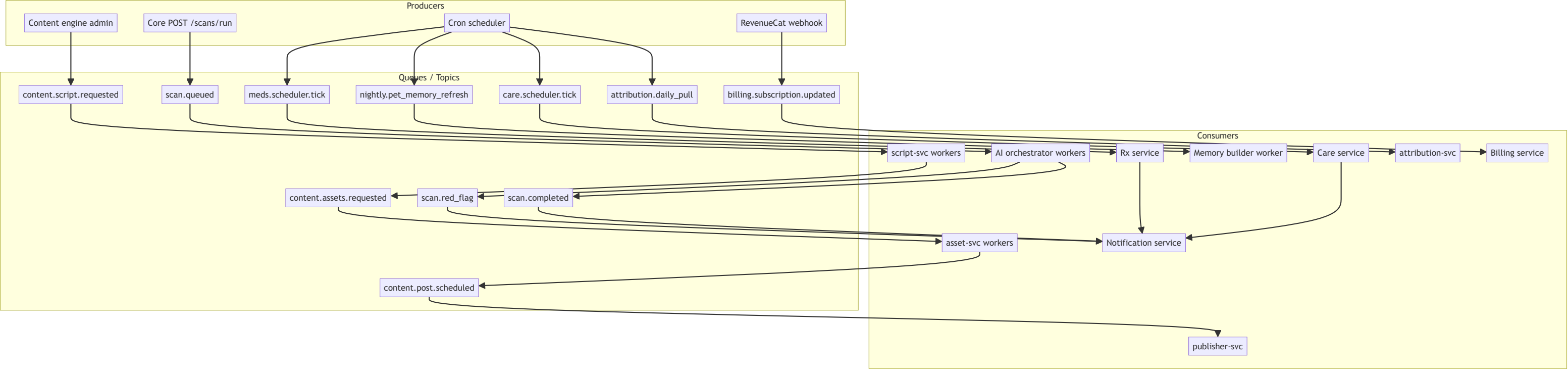
Adds inline citations from the RAG sources that produced the answer.

5 — Disclaimer footer

Server-appended, cannot be removed by the client.

6 — Audit log (ai_invocations)

Records prompt, model, cost, latency, flags. Used for evals and rollback.



Async job & event topology

All non-trivial work is async. Topics named by what happened (past tense for events, .tick for cron-driven, .requested for commands).

TOPICS

scan.queued

A new AI scan is ready to run.

scan.completed

AI finished; downstream consumers create timeline rows and notifications.

scan.red_flag

Separate topic so red-flag handlers (push, on-call dashboards) can subscribe in isolation.

care.scheduler.tick

Cron heartbeat; care service computes which reminders are due.

meds.scheduler.tick

Same pattern for medication doses.

nightly.pet_memory_refresh

Rebuilds per-pet memory chunks from the last 24h of events.

content.script.requested → assets.requested → post.scheduled

The growth-media engine pipeline as three sequential topics.

attribution.daily_pull

Cron-driven pull of social metrics.

billing.subscription.updated

RevenueCat webhook re-emitted onto our bus.

Data model entity relationships (simplified ERD)

The pet is the central entity. Households scope authorisation; AI work (scans, chat) is always logged with an ai_invocations row for audit.

KEY ENTITIES

users / households / household_members

Auth and multi-caregiver scoping. Every authorisation check resolves through this triangle.

pets

The root entity. Almost every other table has a pet_id FK.

scans / scan_inputs / scan_outputs

One scan row plus N input artefacts (photos, voice) and one output payload.

timeline_events

Materialised view of everything that happened to a pet — fed by scans, treatments, notes, meals, care items.

chat_threads / chat_messages

One thread per topic; messages carry an ai_invocations FK if generated.

treatments / treatment_doses / recovery_checkins

Medication schedule plus daily recovery telemetry.

nutrition_plans / meals_logged / foods

Food database (foods) is shared; plans and logs are per pet.

vet_handoffs

Snapshot of the timeline + scans at the time the document was generated.

pet_memory_chunks

Vector-indexed summaries used by the chat RAG flow.

kb_chunks (from education_articles)

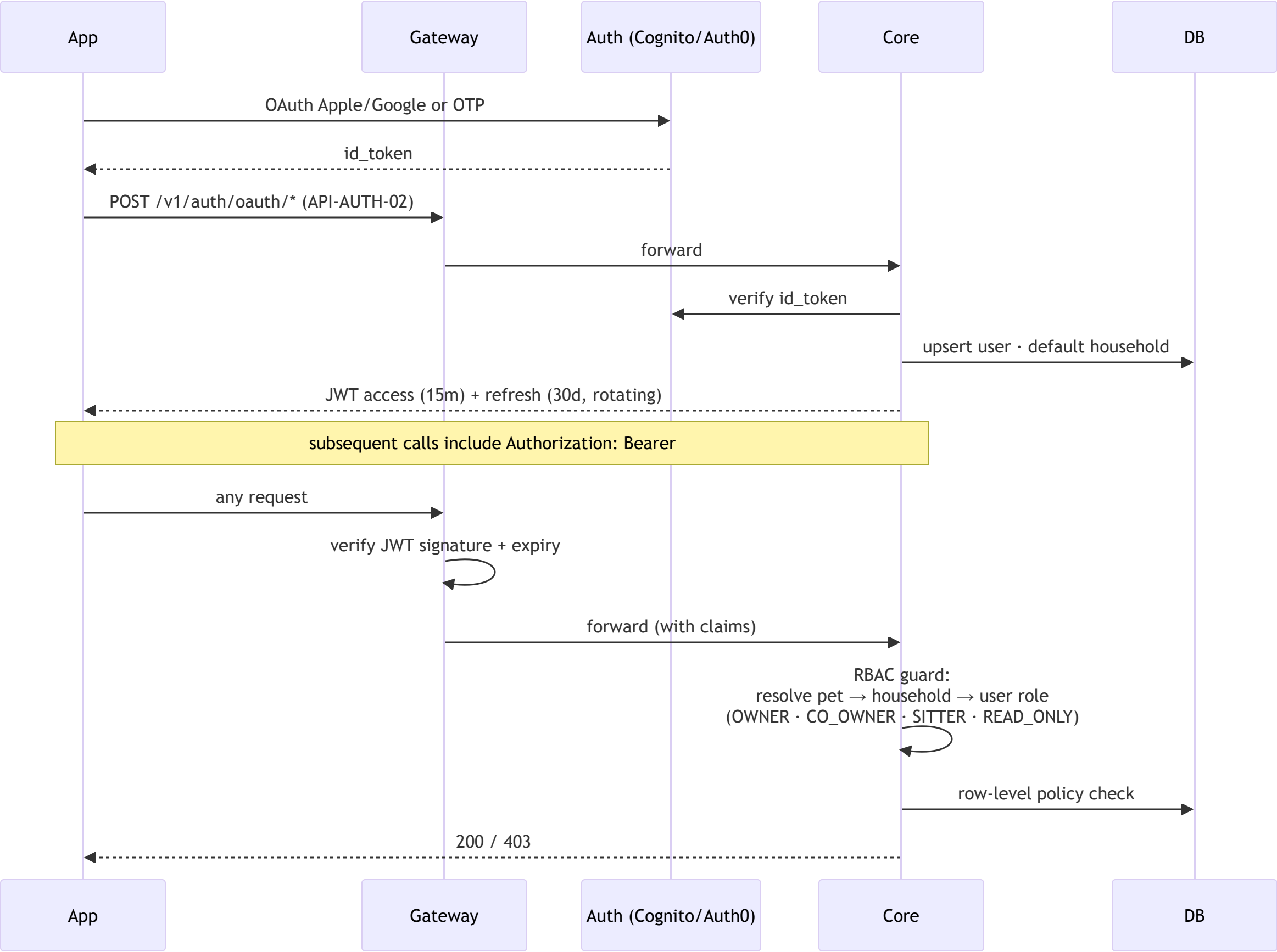
Curated knowledge base used for citations.

ai_invocations

One row per LLM/vision call. Single source of truth for cost, latency, model version, safety flags.

subscriptions / consents / notifications

Cross-cutting tables; not pet-scoped.



Authentication & authorisation flow

OAuth (Apple/Google) or OTP. Short-lived access JWT + rotating refresh. Authorisation is row-level and resolved through household roles.

SEQUENCE

OAuth / OTP at Auth provider

Cognito or Auth0; never touches passwords ourselves.

POST /v1/auth/oauth/* (API-AUTH-02)

Exchanges the id_token for our own JWTs.

Token pair

Access JWT: 15 minutes. Refresh: 30 days, rotating on every use.

Gateway JWT verification

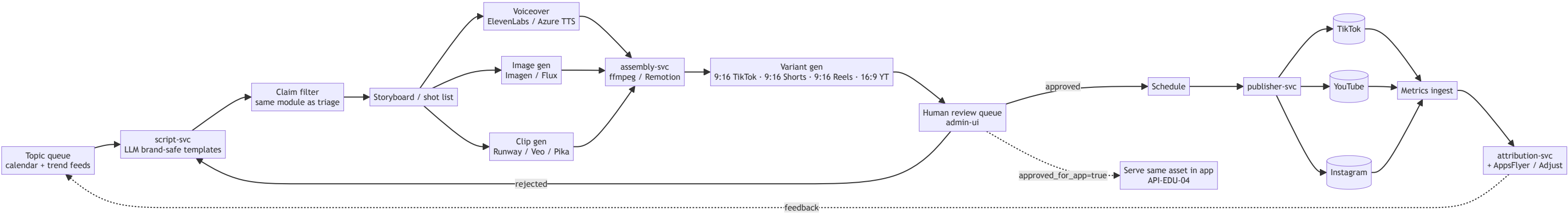
Signature + expiry checked before any request reaches the API.

RBAC guard

Resolves the requested resource → pet → household → user role (Owner / Co-owner / Sitter / Read-only).

Row-level policy

Postgres RLS enforces the same rules at the DB level as a safety net.



Growth-media content engine (Backend SE primary mission)

Pipeline that turns a topic into a posted short video across TikTok / YouTube / Instagram, with a mandatory human review step.

STAGES

Topic queue

Calendar plus trend feeds (Google Trends, TikTok Creative Center).

script-svc

LLM generates a script from brand-safe templates.

Claim filter

Same module as the triage safety pipeline — content cannot make medical claims either.

Storyboard / shot list

Structured plan for the asset stages.

Voiceover (ElevenLabs / Azure TTS)

Stage 1 asset.

Image gen (Imagen / Flux)

Stage 2 asset.

Clip gen (Runway / Veo / Pika)

Stage 3 asset — most expensive, batched.

assembly-svc

ffmpeg / Remotion stitch assets together.

Variant gen

Crops to 9:16 for TikTok / Shorts / Reels, 16:9 for YouTube.

Human review queue (admin-ui)

Required gate; rejection sends back to script-svc with notes.

publisher-svc

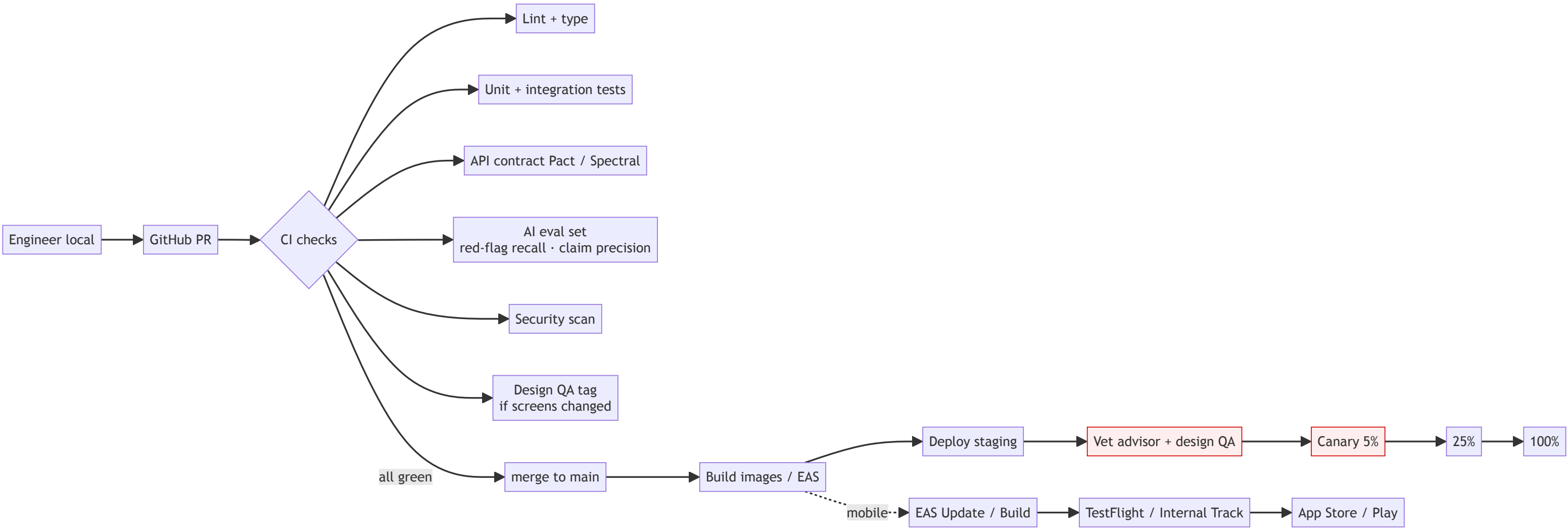
Schedules and posts to each platform.

Metrics ingest → attribution-svc

Pulls performance, joins with install data.

approved_for_app=true

Approved assets are also served inside the app (API-EDU-04) — the same asset becomes both growth and product content.



Environments, CI/CD & release flow

Trunk-based. PRs gated by six checks. Canary rollout (5% → 25% → 100%) after vet advisor + design QA approval.

CI GATES

Lint + type

eslint, prettier, tsc.

Unit + integration tests

Per-service test suites.

API contract (Pact / Spectral)

OpenAPI lint + consumer-driven contracts between mobile and backend.

AI eval set

Recall on red-flag cases, precision on claim filter — hard threshold.

Security scan

SAST + dependency audit.

Design QA tag

Auto-applied when screen-bearing files change; design lead must sign off.

RELEASE PATH

Build images / EAS

Backend → container images. Mobile → EAS build/update.

Staging

Deployed automatically after green CI.

Vet advisor + design QA gate

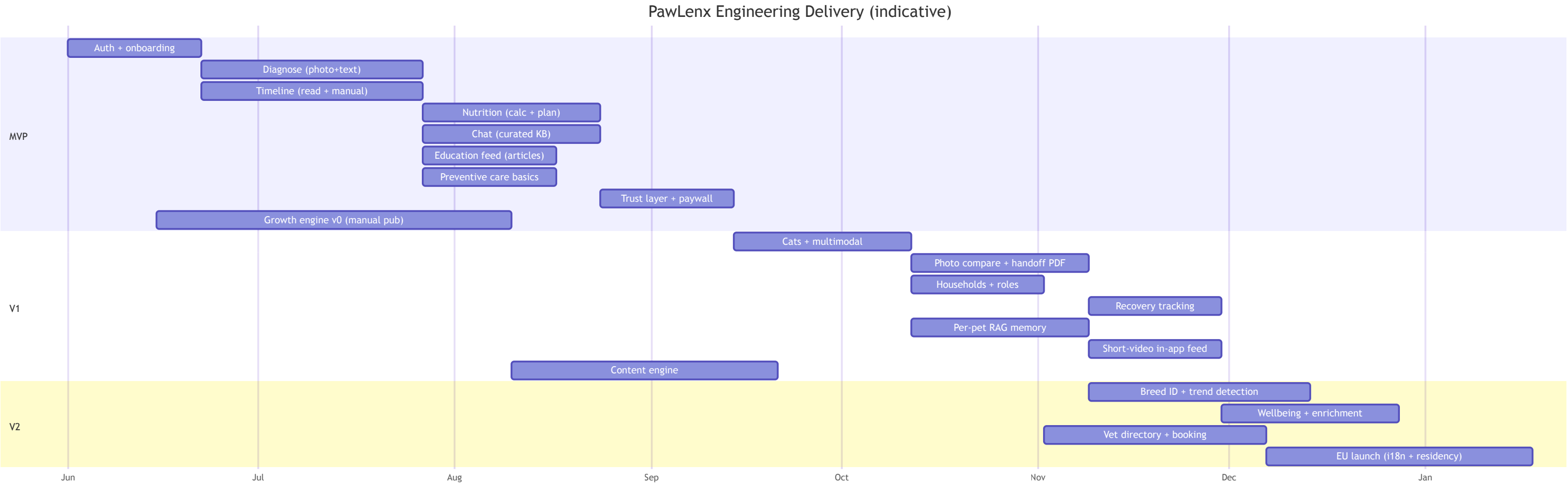
Human approval before production.

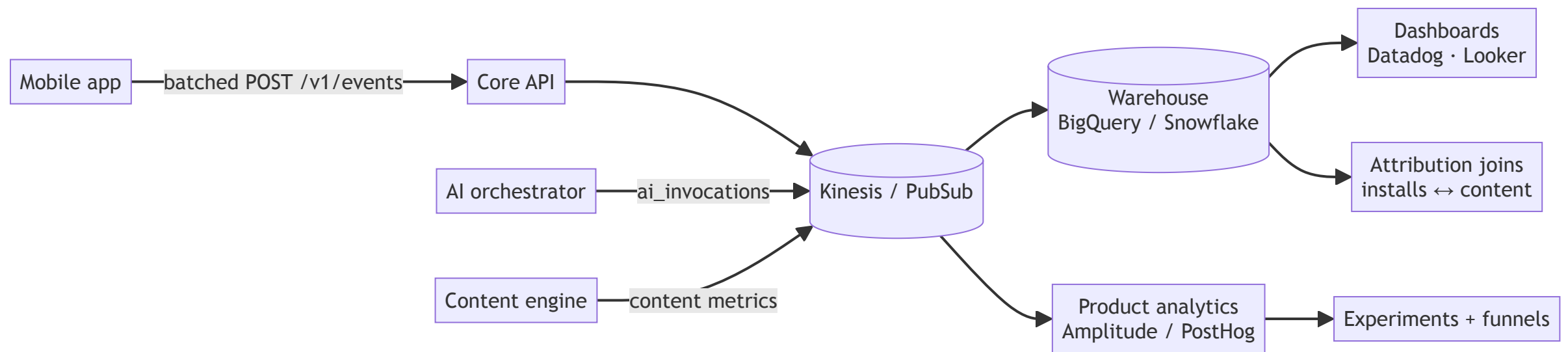
Canary 5% → 25% → 100%

Backend rollout via traffic shifting.

TestFlight / Internal Track → App Store / Play

Mobile release path.





Telemetry pipeline

Single ingest path from app, AI orchestrator and content engine. Stream fans out to warehouse, product analytics and attribution joins.

NODES

Mobile app → POST /v1/events

Batched event uploads; offline-safe queue.

ai_invocations

Streamed from the AI orchestrator the moment a call completes.

Content metrics

Pushed from the publisher and attribution services.

Stream (Kinesis / PubSub)

Buffer + fan-out.

Warehouse (BigQuery / Snowflake)

Long-term store for analysis and ML.

Product analytics (Amplitude / PostHog)

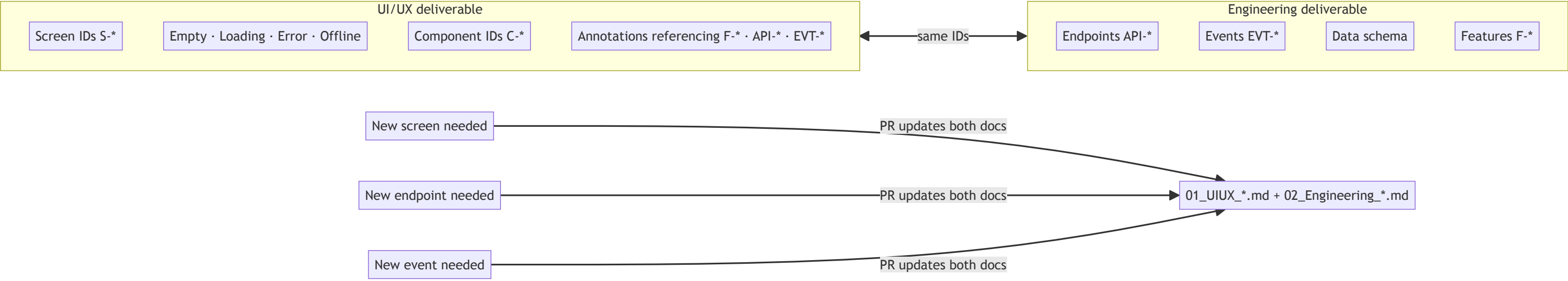
Funnels, retention cohorts, experiments.

Dashboards (Datadog · Looker)

Ops and business dashboards.

Attribution joins

Installs ↔ content piece ↔ retention.



Bidirectional handoff contract (design ↔ engineering)

Process diagram. The two docs (UI/UX and Engineering) are the contract. New screens, endpoints and events require a PR that updates both.

SHARED IDS

Screen IDs (S-*)

Owned by design; referenced by engineering routes and API tests.

Component IDs (C-*)

Owned by design; map to React Native components.

Feature IDs (F-*)

Owned by both; one feature folder per ID in the mobile repo.

Endpoint IDs (API-*)

Owned by engineering; called out in design annotations so flows are testable.

Event IDs (EVT-*)

Owned by engineering; design annotates which screen emits which event.